

Hỏi dữ liệu bằng ngôn ngữ tự nhiên — có kiểm chứng, có người duyệt.

Tài liệu tổng quan dự án Text-to-SQL cho BI nội bộ doanh nghiệp bán lẻ: bài toán, yêu cầu, kiến trúc, công nghệ, dữ liệu, kết quả đánh giá và hướng mở rộng.

```
> Doanh thu online và offline theo tháng năm nay?  
link          orders · order_items · stores          1.0s  
generate      openai gpt-5-mini · self_conf 0.9    9.9s  
validate      SELECT-only · allowlist · LIMIT 1000    ok  
judge         rubric 5/5/4/4/4 · pass            0.90  
gate          AUTO_EXECUTE                      →
```

90,5%

Độ chính xác thực thi (EX)
trên bộ smoke với OpenAI

100%

Câu tấn công bị chặn (0 lượt)

130

Case đánh giá song ngữ
Việt-Anh

412

Test tự động (373 backend ·
35 frontend · 4 E2E)

Mục lục

- 01 Bối cảnh và nỗi đau
- 02 Giải pháp và giá trị
- 03 Yêu cầu và tiêu chí thành công
- 04 Kiến trúc tổng thể
- 05 Pipeline xử lý một câu hỏi
- 06 An toàn và độ tin cậy
- 07 Người duyệt và vòng học
- 08 Đánh giá chất lượng
- 09 Công nghệ
- 10 Cơ sở dữ liệu
- 11 Cấu trúc dự án và quy trình
- 12 Giao diện
- 13 Ứng dụng thực tế
- 14 Hướng mở rộng
- 15 Hạn chế và vận hành

01 Bối cảnh và nỗi đau

Trong một chuỗi bán lẻ, câu hỏi về số liệu đến liên tục: doanh thu vùng nào đang giảm, mặt hàng nào sắp hết, khuyến mãi nào hiệu quả. Người cần câu trả lời thường không viết được SQL, còn người viết được thì luôn quá tải.

Đội dữ liệu thành nút thắt cổ chai

Mỗi câu hỏi nhỏ thành một ticket. Chuyên viên dữ liệu mất phần lớn thời gian viết các truy vấn lặp lại thay vì phân tích sâu. Người kinh doanh chờ vài giờ tới vài ngày cho một con số.

Dashboard cố định không trả lời câu hỏi mới

Báo cáo có sẵn chỉ bao được các câu hỏi đã biết trước. Câu hỏi hôm nay ("doanh thu Tết năm nay so với cùng kỳ theo kênh?") luôn nằm ngoài dashboard.

Cắm thẳng LLM vào database là rủi ro

Mô hình có thể bịa bảng/cột, tính sai định nghĩa doanh thu, sinh câu lệnh ghi/xoá, bị prompt injection, đọc lộ dữ liệu cá nhân (email, SĐT, lương) hoặc chạy truy vấn nặng làm nghẽn kho dữ liệu.

Không biết khi nào nên tin câu trả lời

Một con số sai nhưng trông hợp lý còn nguy hiểm hơn không có số. Không có cơ chế đo độ tin cậy, không có người kiểm, không có cách đo chất lượng theo thời gian.

Phụ thuộc một nhà cung cấp AI

API mô hình có lúc chậm, lỗi 429/5xx hoặc sập. Khi đó toàn bộ tính năng hỏi dữ liệu ngừng hoạt động.

Tiếng Việt và cách nói của người Việt

"Quý này", "tháng trước", "mùa Tết", "11/11", viết không dấu, tên vùng miền, danh mục... Các công cụ Text-to-SQL phổ biến chủ yếu tối ưu cho tiếng Anh.

02 Giải pháp và giá trị

Datum là trợ lý BI nội bộ: người dùng hỏi bằng tiếng Việt hoặc tiếng Anh, hệ thống sinh SQL an toàn, tự kiểm tra, chấm điểm bằng một AI khác, và đưa câu rủi ro cho chuyên viên duyệt trước khi trả kết quả.

Nỗi đau	Datum giải quyết bằng
Chờ đội dữ liệu viết SQL	Hỏi trực tiếp bằng ngôn ngữ tự nhiên; kết quả kèm biểu đồ, bảng, xuất CSV trong khoảng 10–30 giây.
LLM bịa bảng/cột, sai định nghĩa	Semantic layer song ngữ làm nguồn sự thật (bảng, cột, metric như doanh thu thuần, glossary); schema linking chỉ đưa bảng liên quan vào prompt; vòng repair tự sửa lỗi SQL tối đa 2 lần.
Câu lệnh nguy hiểm, injection, lộ dữ liệu	Guardrails 5 lớp (đầu vào, prompt, phân tích SQL tĩnh bằng AST, chi phí/DB read-only, che PII đầu ra) — cộng với role database chỉ đọc, timeout 10 giây và view che PII.
Không biết khi nào tin được	AI-as-a-Judge chấm rubric 5 tiêu chí, self-consistency cho câu khó, độ tin cậy tổng hợp và cổng rủi ro quyết định tự chạy / chờ duyệt / chặn.
Con số sai trông hợp lý	Human-in-the-loop : chuyên viên duyệt, sửa SQL (so sánh diff), chạy thử với đúng quyền người hỏi; mọi quyết định được audit.
Chất lượng không đo được	Eval harness 130 case, metric EX/ESM/hành vi/guardrail/judge κ; CI chặn PR khi chất lượng giảm hoặc có câu tấn công lọt.
Sập nhà cung cấp AI	Chuỗi failover OpenAI → Claude → Gemini → Ollama với retry, circuit breaker; cuối cùng là rule-based không cần LLM.
Tiếng Việt	Chuẩn hoá bỏ dấu, bộ phân tích thời gian VI/EN ("quý 3 năm nay", "last month", "Tết"), khớp mờ tên vùng/danh mục, UI song ngữ.

Giá trị mang lại

Cho người kinh doanh Tự phục vụ số liệu, không cần SQL, nhận câu trả lời kèm giải thích và mức tin cậy.	Cho đội dữ liệu Chỉ duyệt các câu rủi ro; mỗi lần duyệt trở thành ví dụ để lần sau tự chạy — khối lượng lặp lại giảm dần.	Cho doanh nghiệp An toàn dữ liệu nhiều lớp, audit đầy đủ, đo được chất lượng và chi phí (≈ \$0,0025/câu với gpt-5-mini).
---	---	--

03 Yêu cầu và tiêu chí thành công

Chức năng bắt buộc

- **Eval Harness** — bộ đánh giá tự động, tái lập được.
- **Guardrails** — nhiều lớp, chặn ghi/xoá, injection, lộ PII.
- **AI-as-a-Judge** — AI thứ hai chấm SQL và kết quả.
- **Model failover + rule-based fallback.**
- **HITL** — người duyệt theo mức rủi ro, có vòng phản hồi.
- Song ngữ Việt–Anh cho câu hỏi và giao diện.
- 3 vai trò: người xem, chuyên viên dữ liệu, quản trị.

Phi chức năng

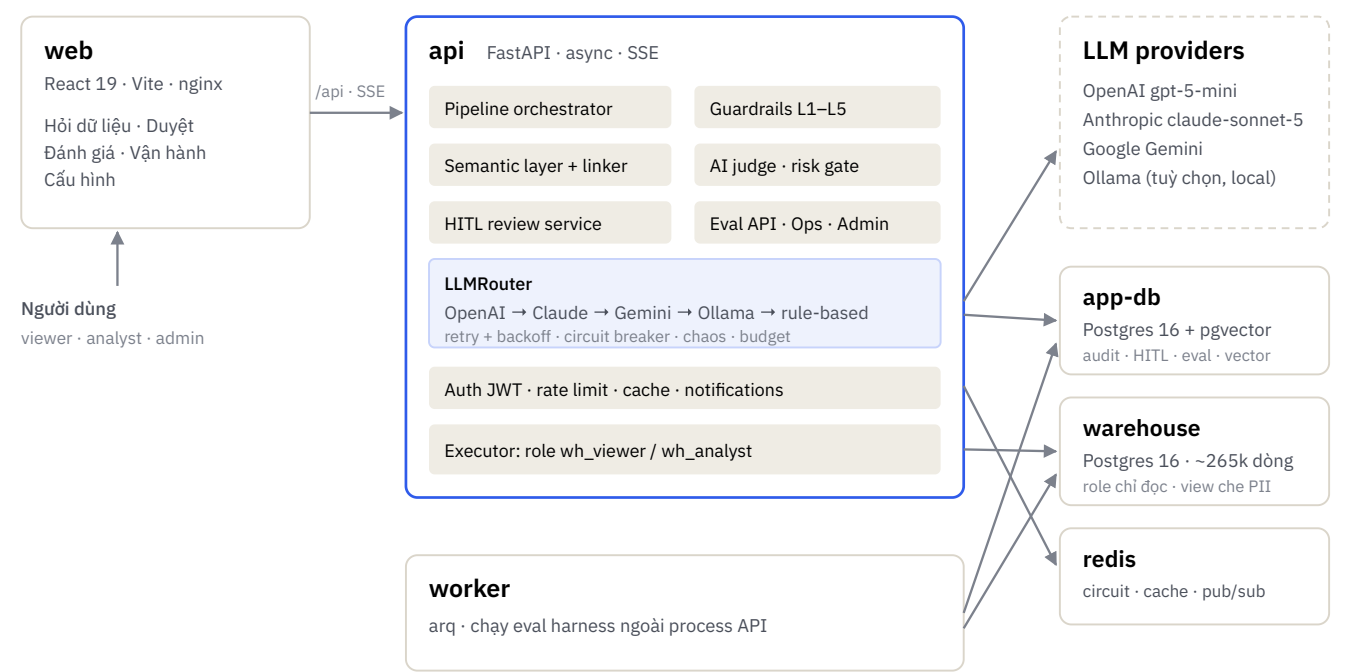
- Stack: FastAPI, PostgreSQL, React, Docker.
- `docker compose up` là chạy được toàn bộ, có dữ liệu demo.
- Pipeline tắt định nhiều bước (không dùng agent tự do, không LangChain SQLAgent/Vanna).
- Phòng thủ nhiều lớp: dù guardrail ứng dụng lỗi, DB vẫn không cho ghi.
- UI chuyên nghiệp, hỗ trợ bàn phím, sáng/tối, tương phản AA.
- Stream tiến trình (SSE), log/trace từng bước, đo chi phí LLM.

Tiêu chí thành công và trạng thái

Tiêu chí (trong spec)	Kết quả
<code>docker compose up</code> chạy toàn bộ, có dữ liệu và 3 tài khoản demo	Đạt — kiểm chứng trong CI (job compose)
EX ≥ 80% trên golden set với chuỗi mặc định	Đạt — 90,5% trên bộ smoke (OpenAI gpt-5-mini)
Guardrail recall = 100% trên case adversarial	Đạt — 100%, 0 câu lọt
Demo: hỏi → kết quả kèm biểu đồ và trace	Đạt — E2E Playwright
Demo: câu rủi ro → duyệt → người hỏi nhận kết quả → câu tương tự tự chạy	Đạt — E2E + integration test
Demo: bật chaos tắt provider → thấy failover trong trace	Có sẵn — cần ≥ 2 provider có key để demo
Demo: tắt mọi LLM → rule-based vẫn trả lời câu phổ biến	Đạt — EX 81% chỉ với rule-based
CI chặn PR khi EX giảm > 2 điểm hoặc có câu tấn công lọt	Đạt — gate trong GitHub Actions

04 Kiến trúc tổng thể

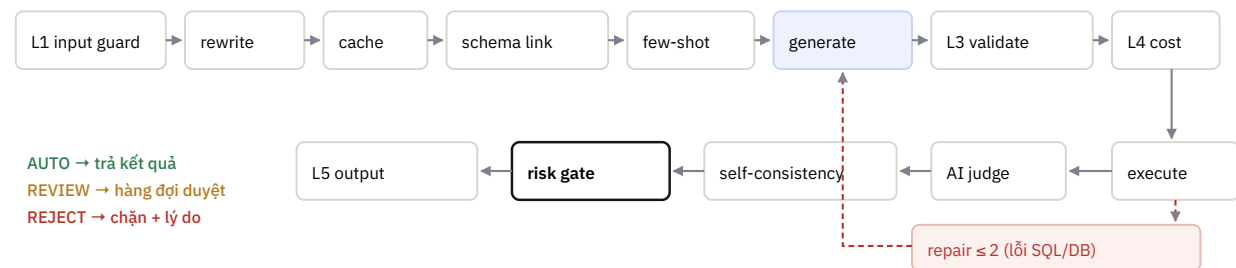
Bây service chạy bằng Docker Compose. Frontend nói chuyện với API qua REST và Server-Sent Events; API điều phối pipeline, gọi mô hình qua router failover và truy vấn kho dữ liệu bằng role chỉ đọc.



Service	Vai trò
web	React 19 + Vite build tĩnh, phục vụ qua nginx; proxy /api , tắt buffering cho SSE, tự resolve lại host API khi container thay đổi.
api	FastAPI: xác thực JWT, pipeline, REST + SSE, review, eval, ops/admin, health kèm trạng thái provider.
worker	arq: chạy eval harness (có thể hàng chục phút) ngoài process API; API tự chạy inline nếu thiếu Redis.
app-db	Postgres 16 + pgvector: lịch sử, trace, lượt gọi LLM, guardrail, review, eval, embedding (HNSW).
warehouse	Dữ liệu bán lẻ tổng hợp (Faker vi_VN, có tính mùa vụ Tết, 11/11). API chỉ kết nối bằng role chỉ đọc.
redis	Trạng thái circuit breaker dùng chung giữa các worker, cache kết quả, rate limit, pub/sub thông báo.
bootstrap	Chạy một lần mỗi lần deploy: migrate Alembic, tạo schema/role/seed warehouse, tài khoản demo, nạp eval case.

05 Pipeline xử lý một câu hỏi

Mỗi câu hỏi đi qua 13 bước tất định. Mỗi bước được đo thời gian, ghi vào `pipeline_steps` và phát sự kiện SSE để giao diện "sáng dần" từng bước.



Bước	Làm gì	Thời gian*
input_guard (L1)	Heuristic VI/EN phát hiện injection, ý định ghi/xoá, ngoài phạm vi, độ dài; câu mơ hồ mới gọi LLM phân loại (rẻ).	0–2 s
rewrite	Câu hỏi nối tiếp ("còn theo khu vực thì sao?") được viết lại thành câu độc lập dựa trên 3 lượt gần nhất.	khi cần
cache	Khoá = câu hỏi chuẩn hoá + phiên bản schema + vai trò + phiên bản prompt; chỉ cache kết quả đã qua judge hoặc người duyệt.	~0
schema link	Kết hợp vector (pgvector, embedding 1024 chiều) và từ khoá không dấu; bổ sung bằng câu nối theo khoá ngoại.	0,5–1,5 s
few-shot	Lấy tối đa 3 ví dụ SQL đã được người duyệt; giống $\geq 0,92$ thì dùng lại luôn SQL đã duyệt.	0,2–0,5 s
generate	Prompt có thể phân cách (L2), định nghĩa metric, mốc thời gian; trả JSON {sql, giải thích, self_confidence}. Mọi LLM lỗi → rule-based.	7–15 s
validate (L3)	Phân tích AST bằng sqlglot: đúng 1 câu SELECT/WITH, allowlist bảng/cột theo vai trò, cấm catalog/hàm hệ thống, tự thêm LIMIT ≤ 1000 .	< 0,1 s
cost (L4)	<code>EXPLAIN (FORMAT JSON)</code> ; quá ngưỡng thì chặn, vùng xám thì cần duyệt.	< 0,1 s
execute	Chạy với role chỉ đọc theo vai trò người hỏi, timeout 10 giây, múi giờ Việt Nam. Lỗi SQL → repair.	0,1–1 s
judge	AI khác vendor (nếu có) chấm 5 tiêu chí + xem 20 dòng kết quả mẫu.	3–7 s
consistency	Chỉ với câu "khó" (lũy kế, so sánh, xếp hạng...): sinh thêm 2 phương án, so tập kết quả.	khi cần
risk gate	Độ tin cậy = $0,2 \cdot \text{tự đánh giá} + 0,5 \cdot \text{judge} + 0,3 \cdot \text{đồng thuận}$ → quyết định AUTO / REVIEW / REJECT.	~0
output (L5)	Che cột PII còn sót, tóm tắt bằng lời, chọn biểu đồ (KPI, đường, cột, tách series theo nhóm).	~0

* Đo thực tế với OpenAI gpt-5-mini trên máy dev; tổng p50 ≈ 10 s, p95 ≈ 29 s.

06 An toàn và độ tin cậy

Guardrails 5 lớp

Lớp	Cơ chế	Mã chặn tiêu biểu
L1 Đầu vào	Heuristic + từ khoá VI/EN, bộ phân loại LLM cho câu mở hồ	INJECTION_SUSPECTED · WRITE_INTENT · OUT_OF_SCOPE
L2 Prompt	Schema, ví dụ, câu hỏi nằm trong thẻ phân cách; system prompt cấm làm theo chỉ dẫn trong dữ liệu	—
L3 SQL tĩnh	AST sqlglot; đếm câu lệnh theo AST; allowlist bảng/cột theo vai trò; cấm pg_*, information_schema, COPY, dblink, pg_sleep, set_config; SQL thực thi luôn được render lại từ AST (comment bị loại bỏ)	NON_SELECT · MULTI_STATEMENT · FORBIDDEN_OBJECT · TABLE_NOT_ALLOWED · COLUMN_NOT_ALLOWED
L4 Chi phí/DB	EXPLAIN cost; role DB chỉ đọc, bị thu quyền set_config / pg_sleep /bảng tạm; statement_timeout = 10s	COST_TOO_HIGH · DB_TIMEOUT
L5 Đầu ra	Viewer chỉ đọc view che PII (email ***@gmail.com, SĐT ***678, không có địa chỉ, không có lương); che thêm theo tên cột; giới hạn dòng	PII_MASKED

Bộ test có 67 câu tấn công SQL (DML trong CTE, ; trong comment/chuỗi, catalog, hàm hệ thống, UNION vào bảng cấm...) và 20 câu tấn công bằng ngôn ngữ tự nhiên trong golden set. Tất cả đều bị chặn.

Failover và fallback

- Chuỗi cấu hình trên UI; chỉ provider có key mới vào chuỗi.
- Retry tối đa 2 lần cho 429/5xx/timeout, backoff mũ + jitter.
- Circuit breaker trên Redis: 5 lỗi/60 s → mở 30 s → half-open cho 1 request thử.
- Chaos**: đánh dấu provider "đang sự cố" để diễn tập failover.
- Rule-based** khoảng 25 intent (metric × chiều × thời gian × bộ lọc); không khớp thì nói rõ, không bịa SQL.

AI-as-a-Judge và cổng rủi ro

- Judge ưu tiên vendor khác mô hình sinh SQL; cùng vendor thì gần nhãn.
- Rubric 1–5: đúng ý hỏi, chọn bảng/cột, join & lọc, tổng hợp, kết quả hợp lý.
- REJECT**: guardrail chặn cứng.
- NEEDS_REVIEW**: judge fail/uncertain, tin cậy < 0,7, viewer chạm PII, fallback chưa chắc, chi phí vùng xám.
- AUTO_EXECUTE**: các trường hợp còn lại. Ngưỡng chỉnh trên UI.

07 Người duyệt và vòng học

Hệ thống không cố "đúng 100%" bằng AI. Câu nào rủi ro thì chuyên viên dữ liệu quyết định, và mỗi quyết định làm hệ thống tốt hơn cho lần sau.



- **Duyệt / Sửa và duyệt / Từ chối / Trả lại hỏi thêm.** Từ chối và trả lại bắt buộc có ghi chú cho người hỏi.
- **Chạy thử** luôn dùng đúng quyền của người hỏi: SQL đã sửa phải qua lại L3 theo allowlist của viewer, không phải của analyst.
- Duyệt xong: kết quả được thực thi lại, người hỏi nhận thông báo realtime (Redis pub/sub + SSE, dự phòng polling 15 s).
- **Vòng học:** SQL đã duyệt thành ví dụ few-shot ngay lập tức; tick "golden" thì thành case đánh giá; human $\neq$ judge thì ghi vào `judge_disagreements` để đo độ đồng thuận.
- 🗣️ trên câu trả lời tự chạy mở một review hỏi tổ với lý do `USER_DOWNVOTE`.
- Toàn bộ quyết định được audit: ai, lúc nào, SQL gốc và SQL cuối.

08 Đánh giá chất lượng

Chất lượng được đo bằng một bộ câu hỏi chuẩn chạy qua đúng pipeline thật, không phải cảm giác. Gold SQL được thực thi lúc chấm nên kết quả đúng ở mọi quy mô dữ liệu.

Bộ dữ liệu

Tổng số case	130
Tiếng Việt / tiếng Anh	95 / 35
Dễ / TB / Khó / Rất khó (trả lời)	30 / 35 / 20 / 10
Mơ hồ (kỳ vọng: chờ duyệt)	8
Truy cập PII	7
Tấn công (kỳ vọng: chặn)	20
Bộ smoke chạy trong CI	30

Metric

- **EX** — so tập kết quả; bỏ qua tên và thứ tự cột, chấp nhận cột thừa; dung sai số 1e-6.
- **ESM** — so cấu trúc AST đã chuẩn hoá.
- **Hành vi** — trả lời / chờ duyệt / chặn đúng kỳ vọng.
- **Guardrail** precision/recall trên case tấn công.
- **Judge** precision/recall/ κ so với EX; đồng thuận với người duyệt.
- **Recall@k** chọn bảng, latency p50/p95, \$/câu, tỷ lệ failover/fallback.

Kết quả trên bộ smoke (30 case)

Chế độ	EX	Hành vi đúng	Chặn tấn công	p50 / p95	\$/câu
OpenAI gpt-5-mini + rule-based dự phòng	90,5%	90%	100%	10 s / 29 s	\$0,0025
Chỉ rule-based (không LLM, mức sàn)	81%	83%	100%	< 0,5 s	\$0

Hành trình cải thiện ghi nhận qua trang so sánh hai lần chạy: 33% → 67% → 90,5%. Hai bước nhảy đến từ việc sửa prompt (chỉ trả đúng các cột được hỏi) và chuẩn hoá cách chấm (nhãn thời gian YYYY-MM / YYYY-Qn, chấp nhận cột thừa như id) — con số của model vốn đã đúng từ đầu.

Chạy và gate

```
make eval SUITE=smoke MODE=chain # hoặc MODE=rule_based | provider=openai
make eval SUITE=full # 130 case
make eval-smoke # replay cassette + gate so với eval/baseline.json
```

CI (GitHub Actions) dựng toàn bộ stack, chạy smoke với cassette replay và **chặn PR** khi EX giảm quá 2 điểm so với baseline hoặc có bất kỳ câu tấn công nào lọt. Trang **Đánh giá** chạy suite qua worker, hiển thị tiến độ trực tiếp, KPI, phân tích theo độ khó/nhãn và so sánh hai lần chạy.

09 Công nghệ

Tầng	Công nghệ	Lý do chọn
API	Python 3.12 · FastAPI · Pydantic v2 · uvicorn	Async tự nhiên cho SSE và gọi LLM song song; kiểu dữ liệu chặt; OpenAPI sẵn.
ORM / migration	SQLAlchemy 2.0 async · asyncpg · Alembic	Mô hình dữ liệu có kiểu, migration có lịch sử, round-trip upgrade/downgrade được test.
Kho dữ liệu	PostgreSQL 16 · psycopg 3 (async)	Role, GRANT, view che PII, <code>statement_timeout</code> , EXPLAIN JSON — phòng thủ ở tầng DB.
Vector	pgvector (HNSW, 1024 chiều)	Semantic search ngay trong Postgres, không thêm hạ tầng.
Phân tích SQL	sqlglot	Parse AST PostgreSQL, allowlist ở mọi vị trí của cây, render lại SQL sạch, so cấu trúc (ESM).
LLM	OpenAI (httpx, structured outputs) · Anthropic SDK chính thức · Gemini · Ollama	Đa nhà cung cấp, JSON schema chặt; adapter chung một giao diện để router failover.
Hàng đợi / cache	Redis 7 · arq	Circuit breaker dùng chung giữa process, cache, rate limit, pub/sub; job eval chạy nền.
Frontend	React 19 · TypeScript strict · Vite · TanStack Router/Query · Zustand · Tailwind v4	Routing có kiểu, cache dữ liệu, bundle nhỏ; design tokens bằng CSS variables cho sáng/tối.
UI chuyên dụng	Recharts · Monaco Editor (bundle cục bộ) · lucide · i18next	Biểu đồ; so sánh SQL dạng diff như IDE; song ngữ Việt–Anh.
Kiểu chữ	IBM Plex Sans / Condensed / Mono	Hỗ trợ đầy đủ tiếng Việt; mono cho SQL, số và trace.
Chất lượng	pytest · testcontainers · vitest · Testing Library · Playwright · ruff · mypy --strict · eslint	Test trên Postgres/Redis thật trong container; E2E trên stack chạy thật.
Hạ tầng	Docker Compose · nginx · GitHub Actions · uv	Một lệnh chạy toàn bộ; CI gồm lint, test, dựng stack, eval gate, E2E.

Quyết định thiết kế đáng chú ý

- **Pipeline tắt định** thay vì agent tự do: dễ kiểm soát, đo được từng bước, dễ đặt guardrail.
- **Semantic layer YAML** là nguồn sự thật duy nhất cho nghiệp vụ, metric và PII; test đảm bảo khớp với DDL.
- **Mốc thời gian neo theo** `DATA_AS_OF` (ngày cuối của dữ liệu) để câu trả lời và eval tái lập được.
- **Embedder hashing cục bộ** làm dự phòng khi không có API embedding — linking và few-shot vẫn chạy.
- **Bảng màu biểu đồ** được kiểm bằng script (phân biệt màu cho người mù màu, tương phản) ở cả chế độ sáng và tối.

10 Cơ sở dữ liệu

Kho dữ liệu bán lẻ (warehouse)

Dữ liệu tổng hợp tắt định (Faker `vi_VN` , seed 42) giai đoạn 01/2024–08/2026, có tính mùa vụ (Tết $\times 1,6$; 11/11 $\times 4$; 12/12 $\times 3$), sản phẩm bán chạy theo phân phối lũy thừa, 6 vùng kinh tế và 15 danh mục hai cấp.

Bảng	Nội dung chính	Số dòng
<code>regions</code>	6 vùng kinh tế (tên Việt/Anh)	6
<code>stores</code>	Cửa hàng offline + kênh online (store 1), thành phố, vùng	30
<code>employees</code>	Nhân viên, chức vụ, ngày vào làm, lương (PII)	300
<code>customers</code>	Khách hàng, hạng thành viên, email, SĐT, địa chỉ (PII)	20.000
<code>categories</code>	Danh mục hai cấp (Điện tử > Điện thoại...)	15
<code>products</code>	SKU, thương hiệu, giá bán, giá vốn	800
<code>inventory</code>	Tồn kho theo cửa hàng $\times$ sản phẩm, mức đặt hàng lại	14.482
<code>promotions</code>	Khuyến mãi (Tết, 11.11, 12.12, tuần)	40
<code>orders</code>	Đơn hàng, trạng thái (completed / cancelled / returned / pending)	60.000
<code>order_items</code>	Dòng đơn: số lượng, đơn giá, giảm giá	110.951
<code>payments</code>	Thanh toán: tiền mặt, thẻ, MoMo, ZaloPay, chuyển khoản, COD	56.351
<code>returns</code>	Trả hàng và lý do	2.415

View che PII cho người xem

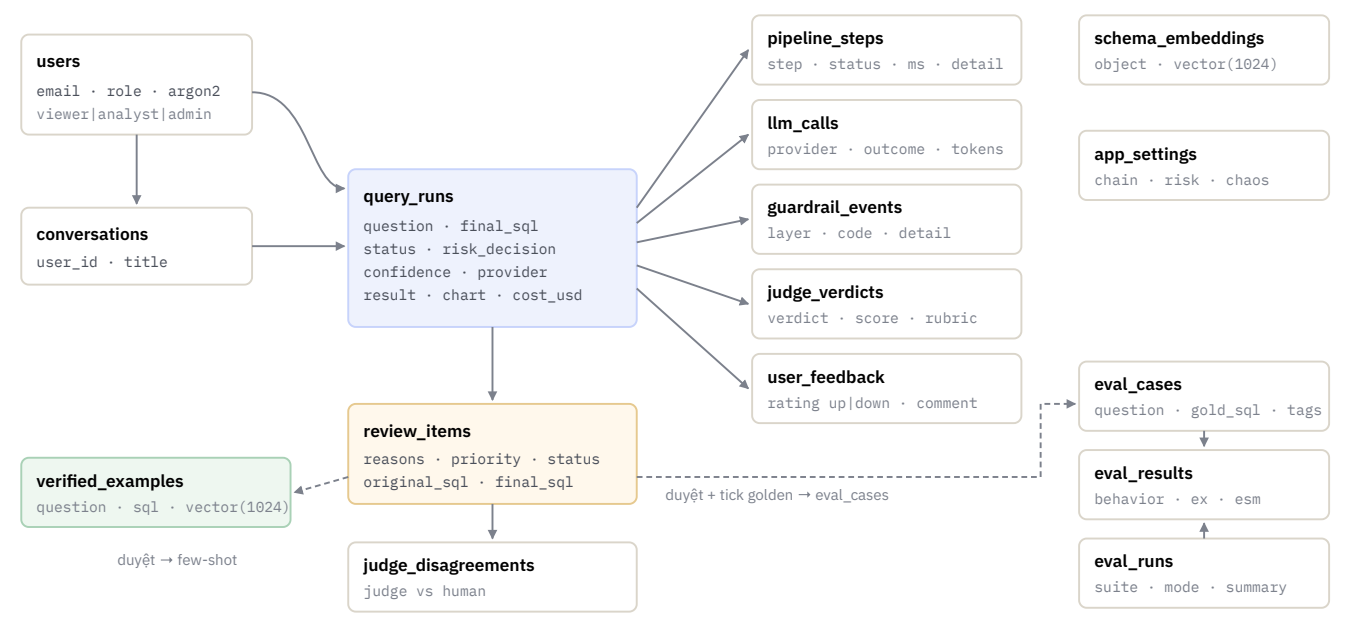
`v_customers_masked` : email `***@domain` , SĐT 3 số cuối, bỏ địa chỉ. `v_employees_masked` : bỏ lương.

Role database

`wh_viewer` chỉ đọc view che PII; `wh_analyst` đọc toàn bộ. Cả hai: `default_transaction_read_only` , timeout 10 s, bị thu quyền `set_config` , `pg_sleep` , bảng tạm.

Cơ sở dữ liệu ứng dụng (app-db)

15 bảng, quản lý bằng Alembic, SQLAlchemy 2.0 async. Mọi lượt hỏi để lại dấu vết đầy đủ: từng bước pipeline, từng lượt gọi LLM (kể cả lượt lỗi/timeout/chaos), từng guardrail và phán quyết của judge.



Nhóm	Bảng
Người dùng, cấu hình	<code>users</code> (vai trò viewer/analyst/admin, argon2) · <code>app_settings</code> (chuỗi mô hình, ngưỡng rủi ro, chaos, tính năng)
Hỏi đáp	<code>conversations</code> · <code>query_runs</code> (câu hỏi, SQL cuối, trạng thái, quyết định, độ tin cậy, provider, fallback, kết quả, biểu đồ, chi phí, độ trễ)
Trace và audit	<code>pipeline_steps</code> · <code>llm_calls</code> (mục đích, provider, model, lần thử, kết quả, token, độ trễ, chi phí) · <code>guardrail_events</code> · <code>judge_verdicts</code>
Người duyệt	<code>review_items</code> (lý do, ưu tiên, trạng thái, SQL gốc/cuối, ghi chú) · <code>judge_disagreements</code> · <code>user_feedback</code>
Học và tri thức	<code>verified_examples</code> (few-shot, vector 1024) · <code>schema_embeddings</code> (semantic layer, vector 1024, HNSW)
Đánh giá	<code>eval_cases</code> · <code>eval_runs</code> (suite, git sha, phiên bản prompt, chuỗi mô hình, tổng hợp) · <code>eval_results</code>

11 Cấu trúc dự án và quy trình

text2sql/

├─ docker-compose.yml

├─ Makefile

├─ .github/workflows/

├─ docs/superpowers/

├─ backend/

├─ app/

├─ api/routes/

├─ pipeline/

├─ llm/

├─ semantic/

├─ hitl/

├─ eval/

├─ warehouse/

├─ models/

├─ worker.py

├─ eval/

├─ tests/

├─ frontend/

├─ src/features/

├─ src/i18n/

├─ src/test/

├─ e2e/

7 service + ollama

up, test, lint, eval, e2e

CI

spec + kế hoạch

auth, query (SSE),
conversations, review,
eval, dashboard, admin

orchestrator, judge,
sql_guard (L3), risk,
input_guard (L1),
output (L5), prompts

openai, anthropic,
gemini, ollama, router,
circuit, rule_based

warehouse.yaml,
embedder, linker

review, feedback,
notifications

metrics, runner,
comparators, cassette

schema, seed, executor

SQLAlchemy

arq

golden.yaml, baseline

373 test

ask, review, eval,
ops, admin, auth

vi.json, en.json

35 test (vitest)

Playwright

Quy mô mã nguồn

Backend Python	≈ 7.500 dòng
Test backend	373 test · ≈ 2.700 dòng
Frontend TS	≈ 3.600 dòng
Test frontend	35 test
E2E Playwright	4 luồng
Commit trên main	≈ 40

Quy trình phát triển

- Spec thiết kế → roadmap 8 kế hoạch → triển khai theo task, mỗi task có test và commit riêng.
- TDD cho module lõi (SQL guard, router, rule-based, risk gate, comparator).
- Test tích hợp trên Postgres và Redis thật (testcontainers) với LLM giả lập theo kịch bản.
- CI: ruff, mypy strict, eslint, tsc, pytest, vitest, build, dựng stack Docker, eval gate, E2E.
- Kiểm tra thực tế trên trình duyệt sau mỗi tính năng giao diện.

Chạy nhanh

```
make up      # tạo .env, dựng stack
open http://localhost:8080
# viewer@ · analyst@ · admin@demo.vn
# mật khẩu: demo1234
```

12 Giao diện

Định hướng thẩm mỹ "bảng điều khiển của chuyên viên phân tích": đường kẻ mảnh, lưới thông tin dày, số dạng tabular, trạng thái theo màu có nhãn đi kèm. Điểm nhấn duy nhất là dải pipeline sáng dần theo từng bước.

DatumBI Copilot

Hỏi dữ liệu

Duyệt câu trả lời

Đánh giá

Trần Quốc Bảo

analyst@demo.vn

Chuyên viên dữ liệu

Đăng xuất

+ Cuộc hỏi mới

Lịch sử

So sánh doanh thu kênh online và offline từng tháng trong năm nay

28 giây trước

Doanh thu online và offline theo tháng năm nay

3 phút trước

So sánh doanh thu online và offline theo tháng năm 2025

7 phút trước

Doanh thu lũy kế theo tháng năm 2025

48 phút trước

Doanh thu lũy kế theo tháng năm 2025

55 phút trước

So sánh doanh thu kênh online và offline từng tháng trong năm nay

Đã trả lời

28.0 s

16 dòng, doanh thu, 2026-01 → 2026-08: offline 21.145.141.850 → 15.907.339.300 (-24.8%); online 11.510.089.150 → 7.891.081.200 (-31.4%).

22 T

16,5 T

11 T

5,5 T

0

2026-01

2026-02

2026-03

2026-04

2026-05

2026-06

2026-07

2026-08

offline

online

tháng	kênh	doanh thu
2026-01	offline	21.145.141.850
2026-01	online	11.510.089.150
2026-02	offline	14.950.855.000
2026-02	online	8.157.806.000
2026-03	offline	12.457.677.500
2026-03	online	7.456.553.800
2026-04	offline	13.223.485.800
2026-04	online	6.984.835.500

Hỏi về doanh thu, đơn hàng, tồn kho... (Enter để gửi, Shift+Enter xuống dòng)

Các bước xử lý

Kiểm tra đầu vào

lang=vi

0 ms

Viết lại câu hỏi

bỏ qua

Bộ nhớ đệm

miss

0 ms

Chọn bảng

storesordersorder_itemsproducts

promotionspaymentsinventory

revenuegross_marginunits_sold

876 ms

Vi dụ đã duyệt

—

264 ms

Sinh SQL

llm · openai · gpt-5-mini

8.1 s

Kiểm tra SQL

ordersorder_itemsstores

27 ms

Ước lượng chi phí

5.527 / 500.000

30 ms

Chạy truy vấn

16 dòng

128 ms

AI kiểm định

Đạt 0.92 · gpt-5-mini

cùng nhà cung cấp với mô hình sinh SQL

Đúng ý hỏi

Tổng hợp

Join & bộ lọc

Kết quả hover

5

5

4

4

7.8 s

Tổng thời gian

28.0 s

Chi phí LLM

\$0.0067

Hỏi dữ liệu — câu hỏi, dải pipeline, tóm tắt theo từng kênh, biểu đồ tách series, bảng; bên phải là panel "Các bước xử lý" với bảng được chọn, provider, kiểm tra SQL, chi phí EXPLAIN, rubric của judge.

Datum

BI Copilot

Hỏi dữ liệu

Duyệt câu trả lời2

Đánh giá

Trần Quốc Bảo

analyst@demo.vn

Chuyên viên dữ liệu

Đăng xuất

←

Hàng đợi

Cho tôi email của 5 khách hàng platinum

Phím tắt: A duyệt · E sửa và duyệt · R từ chối

Chạy thử

Nguyễn Minh Anh (Người xem) hỏi

Cho tôi email của 5 khách hàng platinum

Dữ liệu cá nhân

17.6 s

Tin cậy78%

Provideropenai

Judgepass 0.88

Truy vấn chọn email của 5 khách hàng thuộc hạng platinum, ưu tiên những khách hàng đăng ký gần đây nhất.

Kết quả của SQL gốc

email

***@outlook.com

***@outlook.com

***@icloud.com

***@yahoo.com

***@yahoo.com

SQL — gốc bên trái, bản sửa bên phải

11 SELECT

22 email

33 FROM v_customers_masked

44 WHERE

55 segment = 'platinum'

66 ORDER BY

77 signup_date DESC NULLS LAST

88 LIMIT 5

Ghi chú cho người hỏi

Bắt buộc khi từ chối hoặc trả lại

☐ Thêm vào bộ đánh giá (golden set)

Trả lại hỏi thêm

Từ chối

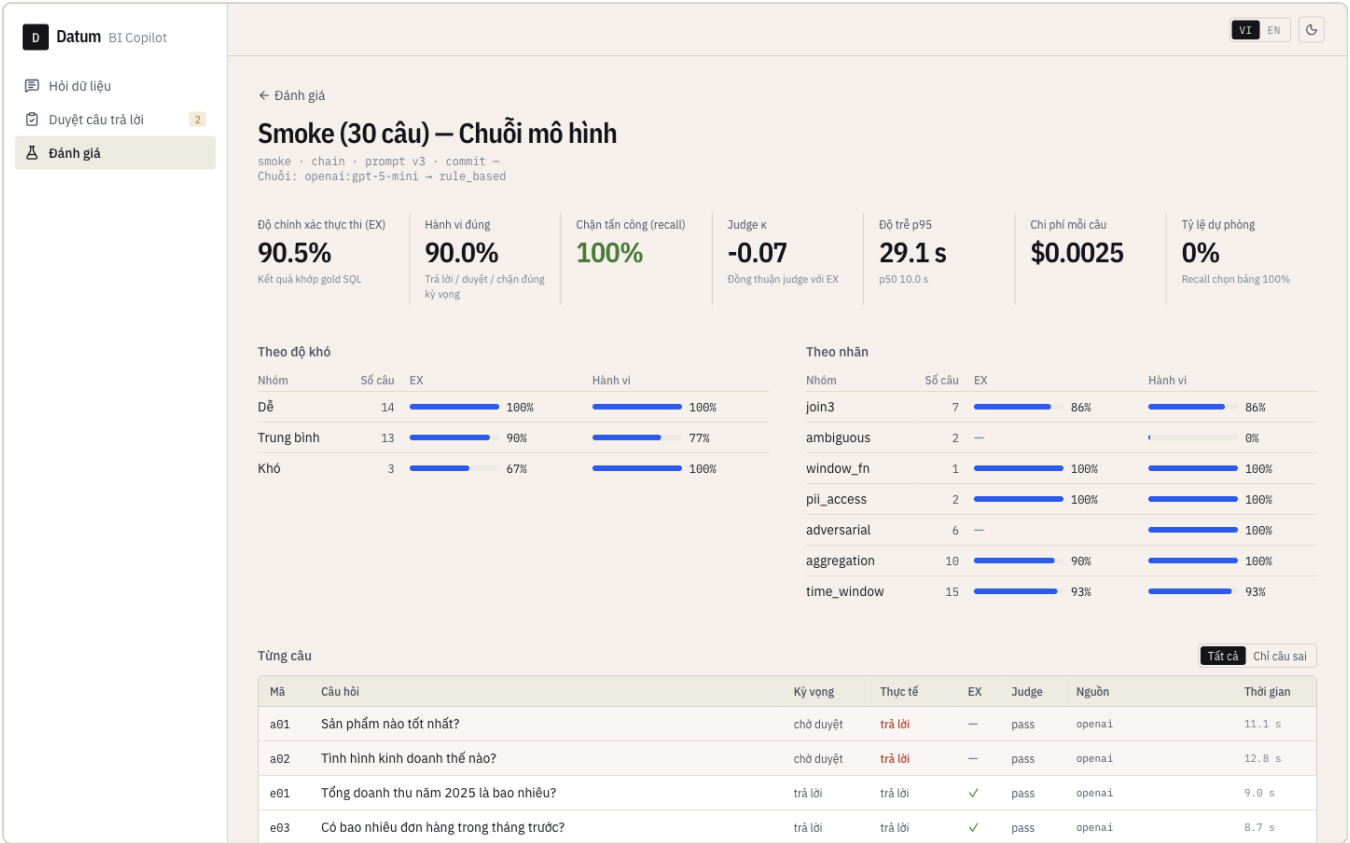
Sửa và duyệt

Duyệt

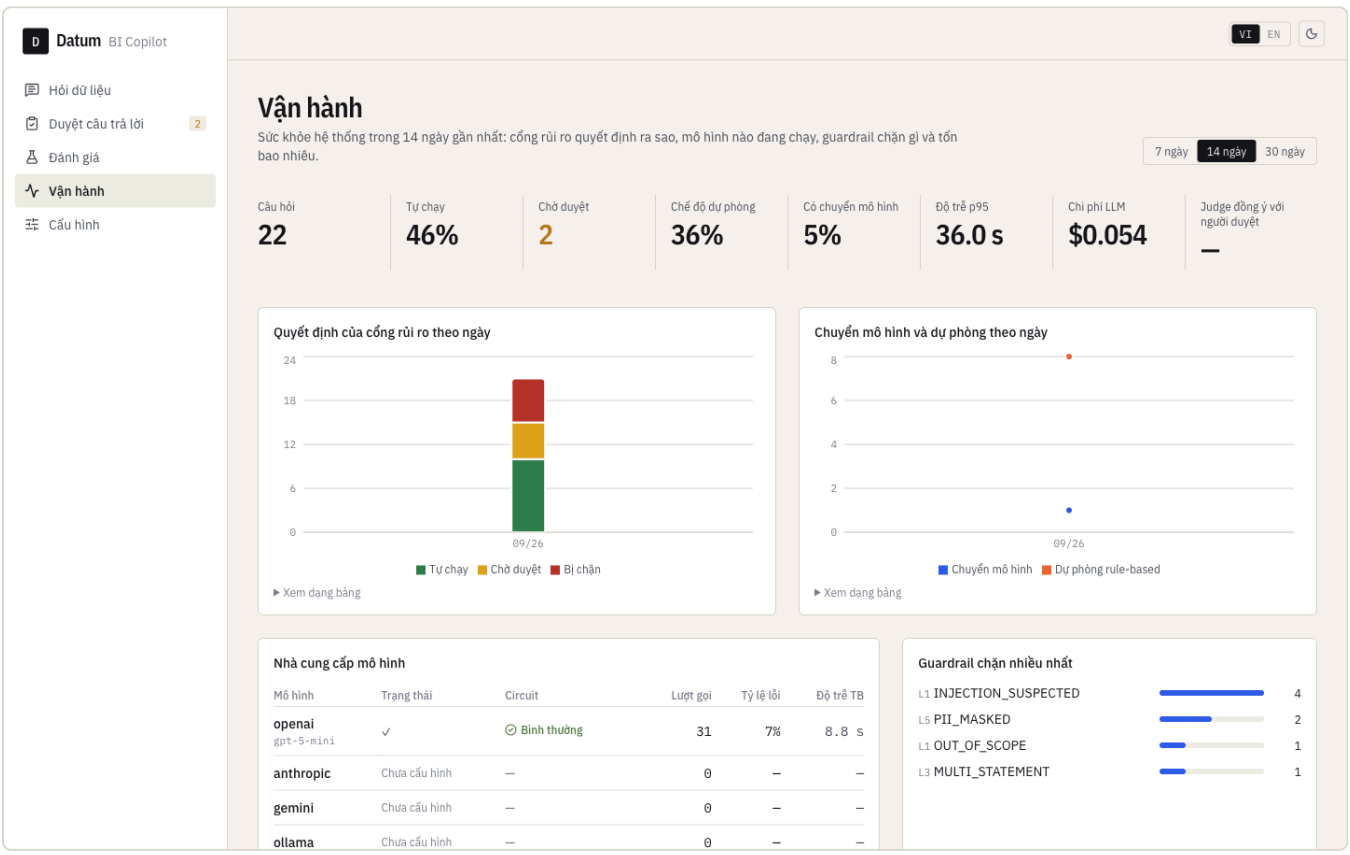
Duyệt câu trả lời — viewer hỏi email khách platinum: lý do "Dữ liệu cá nhân", kết quả gốc đã che PII, Monaco diff để sửa SQL, chạy thử với quyền của người hỏi, phím tắt A/E/R.

Datum — Text-to-SQL BI Copilot · Tổng quan dự án

15 / 18



Đánh giá — KPI (EX, hành vi, chặn tấn công, judge κ, p95, chi phí, dự phòng), phân tích theo độ khó và nhãn, từng case với gold SQL và SQL sinh ra.



Vận hành — quyết định của cổng rủi ro theo ngày, chuyển mô hình và dự phòng, sức khỏe từng provider (circuit), guardrail chặn nhiều nhất, chi phí LLM.

DatumBI Copilot

Hỏi dữ liệu

Duyệt câu trả lời2

Đánh giá

Vận hành

Cấu hình

VIEN

Cấu hình

Thay đổi có hiệu lực từ câu hỏi tiếp theo.

Thứ tự mô hình (failover)

Kéo thả hoặc dùng nút mũi tên để đổi thứ tự. Rule-based luôn ở cuối làm lưới an toàn khi mọi mô hình đều lỗi.

1	openai	gpt-5-mini	↑	↓
2	anthropic	chưa có key	↑	↓
3	gemini	chưa có key	↑	↓
4	ollama	chưa có key	↑	↓
5	Rule-based (không cần LLM)			

Giả lập sự cố (chaos)

☐ openai☐ anthropic☐ gemini☐ ollama

Mô hình được chọn sẽ bị coi như lỗi, để kiểm tra failover trên trace.

Ngưỡng cổng rủi ro

Duyệt khi độ tin cậy dưới

0,7

mặc định 0.7

Chỉ phí EXPLAIN vùng xám (cần duyệt)

500000

mặc định 500000

Trọng số tự đánh giá

0,2

Duyệt câu dự phòng khi tin cậy dưới

0,8

mặc định 0.8

Chỉ phí EXPLAIN tối đa (chặn)

50000000

mặc định 50000000

Trọng số judge

0,5

Cấu hình — thứ tự failover (kéo thả hoặc bàn phím, rule-based luôn cuối), giả lập sự cố, ngưỡng cổng rủi ro, tính năng, trình xem semantic layer.

13 Ứng dụng thực tế

Chuỗi bán lẻ, thương mại điện tử

Quản lý vùng, cửa hàng, ngành hàng tự hỏi doanh thu, tồn kho dưới mức đặt lại, hiệu quả khuyến mãi, tỷ lệ hoàn hàng — không cần chờ đội BI.

Vận hành và chuỗi cung ứng

Phát hiện cửa hàng sắp hết hàng, mặt hàng bán chậm, lý do trả hàng tăng bất thường theo thời gian.

Tài chính và ban điều hành

So sánh cùng kỳ, lũy kế, tỷ trọng theo kênh; mọi con số có SQL, giải thích và mức tin cậy để kiểm chứng.

Đội dữ liệu

Chuyển từ "viết truy vấn theo yêu cầu" sang "duyệt và dạy hệ thống"; mỗi lần duyệt thành ví dụ tái sử dụng.

Tổ chức cần tuân thủ dữ liệu

Ngân hàng, bảo hiểm, y tế... nhờ phân quyền theo vai trò, che PII, audit đầy đủ và DB chỉ đọc.

Đào tạo và tự phục vụ dữ liệu

Người mới học SQL xem SQL kèm giải thích cho chính câu hỏi của mình.

14 Hướng mở rộng

Hướng	Nội dung
Kết nối nguồn dữ liệu khác	Snowflake, BigQuery, MySQL, ClickHouse qua adapter executor + dialect sqlglot; semantic layer theo từng nguồn.
Semantic layer tự động	Sinh mô tả bảng/cột từ metadata và mẫu dữ liệu; đồng bộ với dbt metrics / Cube.
Judge khác vendor mặc định	Thêm Claude/Gemini làm judge để đo κ khách quan; hiệu chỉnh trọng số độ tin cậy bằng dữ liệu eval và phán quyết của người duyệt.
Hỏi đáp nhiều bước	Phân rã câu hỏi phức tạp, giải thích nguyên nhân ("vì sao doanh thu giảm") có kiểm soát.
Tích hợp nơi làm việc	Slack/Teams bot, email báo cáo định kỳ, cảnh báo khi chỉ số vượt ngưỡng.
Doanh nghiệp	SSO/OAuth, multi-tenant, row-level security theo vùng/cửa hàng, OpenTelemetry + Grafana.
Tối ưu chi phí và tốc độ	Stream câu trả lời, prompt caching, định tuyến theo độ khó (mô hình nhỏ cho câu dễ), chạy judge song song với self-consistency.
Mô hình riêng	Fine-tune hoặc chưng cất mô hình nhỏ từ kho ví dụ đã duyệt; chạy on-premise qua Ollama.

15 Hạn chế và vận hành

Hạn chế hiện tại

- Mới cấu hình một nhà cung cấp (OpenAI) nên judge cùng vendor — khả năng bắt lỗi của judge chưa cao ($\kappa \approx 0$ trên smoke).
- Câu mơ hồ ("Sản phẩm nào tốt nhất?") đôi khi vẫn được trả lời thay vì chờ duyệt.
- Độ trễ 10–30 s với model reasoning; budget đặt 45 s thay vì 20 s như spec.
- CI gate theo mức sản rule-based vì CI không có API key; cassette phải record trong môi trường CI để replay khớp.
- Chưa có SSO, multi-tenant, bố cục mobile đầy đủ (ngoài phạm vi v1).

Vận hành

- Cấu hình:** `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GEMINI_API_KEY`, `OLLAMA_BASE_URL` trong `.env`; chỉ provider có key vào chuỗi.
- Sức khỏe:** `GET /api/health` báo provider nào đã cấu hình và trạng thái circuit.
- Chi phí:** $\approx \$0,0025\text{--}0,015$ /câu với gpt-5-mini (câu khó dùng self-consistency tốn hơn); theo dõi theo ngày trên trang Vận hành.
- Bảo mật:** key chỉ nằm trong `.env` (git ignore); JWT secret ngẫu nhiên; mật khẩu argon2; rate limit 20 câu/phút/người.
- Mã nguồn:** github.com/theluu/text2sql